Logical Volume Manager

Imagine you have a Linux server with a single hard disk. Everything is stored on that disk, your operating system, application files, databases, logs, and user data. Initially, everything works perfectly because there is plenty of available storage.

Then time comes, users begin reporting that the application is no longer functioning correctly. Some users cannot upload files, while others receive errors whenever they try to save data. Upon checking the server, you notice several services have failed because they are unable to write new data.

You begin troubleshooting by examining the system logs, application status and finally the disk usage.

The output immediately reveals the problem: the filesystem is at 100% utilization. Since there is no remaining free space, applications cannot create new files, logs cannot be written, and some services may even fail to start.

To resolve the issue, you install an additional hard drive into the server. However, another problem appears. Linux treats the new drive as a completely separate storage device. You cannot simply combine it with the existing disk to create one larger filesystem without performing complex migration or repartitioning.

This is where Logical Volume Manager (LVM) becomes valuable.

LVM introduces a layer of abstraction between the physical disks and the filesystems. Instead of managing storage directly on physical disks or partitions, LVM pools one or more physical storage devices into a single storage pool. From that pool, you can create logical volumes that behave like regular disk partitions but are far more flexible.

With LVM, expanding storage becomes much simpler. Instead of migrating data to a larger disk, you can add another physical disk to the storage pool and extend the logical volume with minimal downtime. This flexibility makes LVM a common choice for servers where storage requirements are expected to grow over time.

The Three Components of LVM

LVM is built around three core components that work together to provide flexible storage management:

1. Physical Volume (PV)
2. Volume Group (VG)
3. Logical Volume (LV)

Physical Volume (PV)

This is the first building block of LVM. It represents a physical storage device or a partition on that device that has been initialized for use by LVM.

Examples of Physical Volumes include:

- An entire hard disk (e.g., /dev/sda)
- An SSD (e.g., /dev/nvme0n1)
- A partition (e.g., /dev/sdb1)

Before LVM can use a disk, it must first be converted into Physical Volume. Once initialized, LVM can recognize and manage the storage space on that device.

Think of a Physical Volume as raw building material. On its own, it simply provides storage capacity but is not yet organized into usable storage for applications.

Volume Group (VG)

This is another component that is needed by LVM. It is a collection of one or more Physical Volumes. Instead of managing each disk individually, LVM combines multiple Physical Volumes into a single storage pool known as a Volume Group. The operating system no longer sees separate disks. It sees one large pool of available storage.

For example:

- Disk 1 = 100 GB
- Disk 2 = 200 GB

After adding both disks to the same Volume Group, LVM presents them as a single storage pool with approximately 300 GB of usable capacity.

This pooled storage is one of LVM's greatest strengths. As storage requirements increase, additional Physical Volumes can be added to the Volume Group, allowing the storage pool to grow without replacing existing disks.

Think of a Volume Group as a warehouse. Individual trucks (Physical Volumes) deliver materials into the warehouse, and the warehouse stores everything in one central location.

Logical Volume (LV)

This is the logical storage allocated block from a Volume Group. Although the Volume Group may contain hundreds of gigabytes or even terabytes of storage, applications do not use the Volume Group directly. Instead, administrators create one or more Logical Volumes from the available storage.

Each Logical Volume behaves much like a traditional disk partition. A filesystem (such as ext4 or XFS) is created on the Logical Volume, it is mounted to a directory, and applications store their data there.

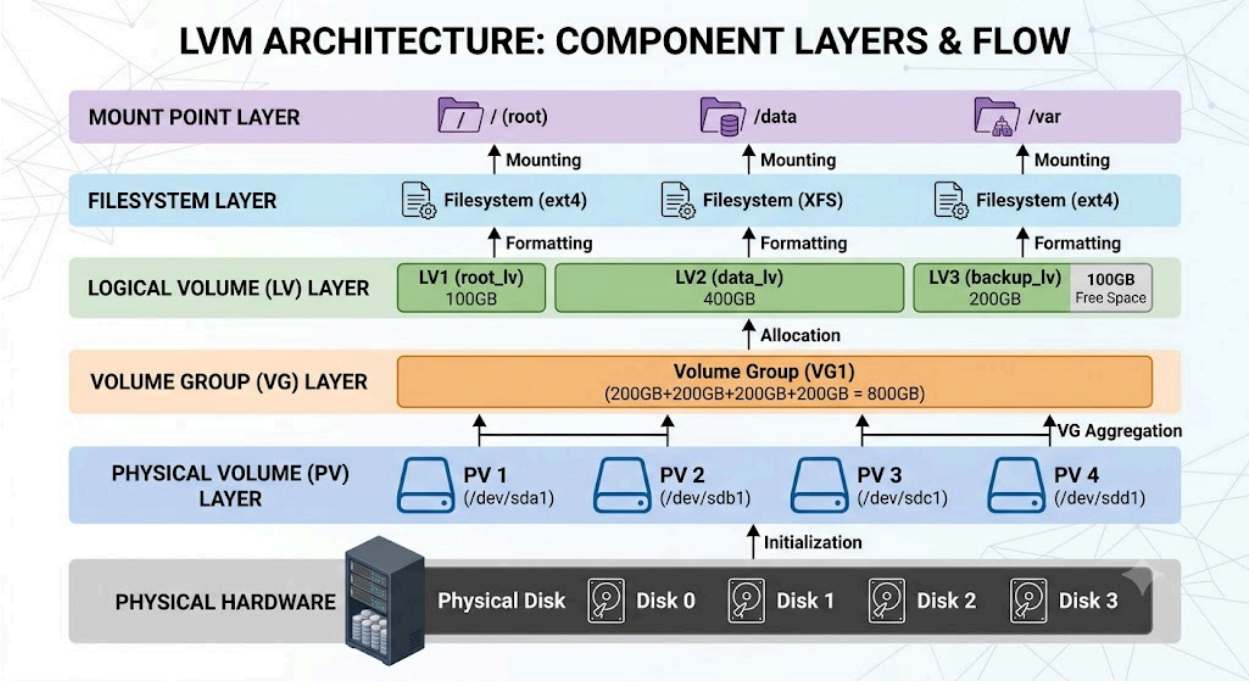
For example, a 300 GB Volume Group could be divided into:

100 GB for application data
150 GB for databases
50 GB for log files

Each of these would be a separate Logical Volume, even though they all share the same underlying storage pool.

One of the major advantages of LVM is that Logical Volumes can often be expanded later without repartitioning the disk or reinstalling the operating system, provided sufficient free space exists in the Volume Group.

Think of Logical Volumes as rooms inside the warehouse. The warehouse contains all available storage, while each room reserves a portion of that storage for a specific purpose.



Hands-On Lab

Objective

Learn how Logical Volume Manager (LVM) abstracts physical storage, allowing flexible storage management without repartitioning disks.

This lab covers:

- Physical Volumes (PV)
- Volume Groups (VG)
- Logical Volumes (LV)
- XFS Filesystems
- Persistent Mounting using /etc/fstab
- Online Logical Volume Expansion
- Expanding a Volume Group by adding a second disk

Prerequisite

OS:
CentOS Stream 9

Virtual Machine:
VMware Workstation Pro

Resources

2 vCPU
4 GB RAM

Storage

60 GB NVMe (Operating System)
30 GB SATA Disk (LVM)
30 GB SATA Disk (Volume Group Expansion)

Phase 1. Adding Two Virtual Hard Disk on a Virtual Machine

In this phase, two additional virtual hard disks will be attached to the CentOS virtual machine.

These disks will not be used as regular partitions. Instead, they will be initialized as Physical Volumes (PVs) that will be managed entirely by LVM.

The first disk will be used to create the initial LVM storage, while the second disk will be added later to demonstrate how a Volume Group can be expanded without affecting the existing filesystem or user data.

After adding the disks we can verify through the shell terminal, we can use the command **lsblk** to see the hierarchy of each physical disks.

```
[centos@localhost ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda           8:0    0   30G  0 disk
sdb           8:16   0   30G  0 disk
nvme0n1     259:0    0   60G  0 disk
├─nvme0n1p1 259:1    0    1G  0 part /boot
└─nvme0n1p2 259:2    0   59G  0 part
  ├─cs-root 253:0    0   37G  0 lvm  /
  ├─cs-swap 253:1    0  3.9G  0 lvm  [SWAP]
  └─cs-home 253:2    0 18.1G  0 lvm  /home
[centos@localhost ~]$
```

The nvme0n1 is an SSD serving as the primary physical disk, we can see the partitions of nvme0n1 and some logical volumes that are created and were mounted.

- The nvme0n1p1 is a partition used for the **/boot** filesystem.
- The nvme0n1p2 was Initialized as an LVM Physical Volume during the operating system installation. there are three logical volume blocks that are managed by LVM. each of them are mounted on a root directory (**/**), the other served as a swap and lastly the other block is used for the /home directory.

These exists as soon as you installed the operating system.

As we also observed that there are two empty Physical disks named as **sda** and **sdb.** We can see that the size is 30G meaning this is the virtual hard disk that we additionally created on a Virtual Machine. We'll be using these to create a Logical Volume Manager.

Phase 2. Creating a Physical Volume

Now that the additional disks have been detected by the operating system, the next step is to initialize one of them as a **Physical Volume (PV).**

A Physical Volume is a disk or partition that has been prepared for use by LVM. Before a disk can participate in a Volume Group, it must first be initialized as a Physical Volume.

Let's verify if there are Physical Volume created. The command to use to verify is:

sudo pvs.

Note: The **sudo** command is used to be able to run the command as root.

```
[centos@localhost ~]$ sudo pvs
  PV             VG Fmt  Attr PSize   PFree
  /dev/nvme0n1p2 cs lvm2 a--  <59.00g    0
[centos@localhost ~]$
```

The sudo pvs command outputs a list of Physical Volume, for now we only have one which come from nvme0n1 physical disk. During the OS installation, the nvme0n1 already had two partition and nvme0n1p2 was one of the partition that is already been created as Physical Volume.

So remember that we added two virtual sata disks. These two are detected correctly by CentOS, but they are not yet initialized as Physical Volume. So let's configure first **sda.** The command is **sudo pvcreate /dev/sda**

Once executed we'll see a message saying **"Physical volume "/dev/sda" successfully created"**

Let's verify:

```
[centos@localhost ~]$ sudo pvcreate /dev/sda
  Physical volume "/dev/sda" successfully created.
[centos@localhost ~]$ sudo pvs
  PV             VG Fmt  Attr PSize   PFree
  /dev/nvme0n1p2 cs lvm2 a--  <59.00g     0
  /dev/sda          lvm2 ---   30.00g 30.00g
[centos@localhost ~]$
```

As we created a Physical volume for the sda, we see a new list on the pvs command. We can see a PV /dev/sda, this means that the LVM now manages a new sda as Physical Volume. We can also check the Size and the Free space for the sda. In this case, we are ready to create a Volume Group for the sda.

Phase 3. Adding a Physical Volume to a Volume Group

In the previous phase, we initialized **/dev/sda** as a Physical Volume (PV). Although the disk is now managed by LVM, it cannot yet be used to store data because a Physical Volume only provides raw storage capacity.

The next step is to create a Volume Group (VG)

A Volume Group combines one or more Physical Volumes into a single storage pool from which Logical Volumes can later be created. Instead of managing individual disks separately, administrators manage the available storage through the Volume Group.

In this phase, we will create a new Volume Group named **vg_lab** using the Physical Volume created in the previous phase.

The **vgcreate** command is used to create a new Volume Group.

sudo vgcreate vg_lab /dev/sda

And just like that, we can also verify using **sudo vgs.**

```
[centos@localhost ~]$ sudo vgcreate vg_lab /dev/sda
[sudo] password for centos:
  Volume group "vg_lab" successfully created
[centos@localhost ~]$ sudo vgs
  VG     #PV #LV #SN Attr   VSize   VFree
  cs       1   3   0 wz--n- <59.00g      0
  vg_lab   1   0   0 wz--n- <30.00g <30.00g
[centos@localhost ~]$
```

We can see that we have two sets of Volume Group, and we can focus on the **vg_lab.** Noticed that there is already one Physical Volume on the **#PV** column. The moment we added **sda** to the vg_lab, the volume size also shows that the size is 30G. This confirms that the **/dev/sda** is added to the vg_lab.

Phase 4. Allocate storage as a Logical Volumes

The Volume Group now provides a pool of available storage. However, the applications cannot use a Volume Group directly. Storage must first be allocated as a **Logical Volume (LV).**

A Logical Volume behaves similarly to a traditional disk partition, except that it resides within a Volume Group and can often be resized later without repartitioning the disk.

In this phase, we will create a 5 GB Logical Volume named lv_data and lv_logs from the Volume Group vg_lab.

The lvcreate command is used to allocate storage from a Volume Group

sudo lvcreate -L 5G -n lv_data vg_lab
sudo lvcreate -L 5G -n lv_logs vg_lab

Whereas:

- **-L 5G** → To allocate 5 GB
- **-n lv_data** → Name of the LV
- **vg_lab** → Which Volume Group to allocate from

```
[centos@localhost ~]$ sudo lvcreate -L 5G -n lv_data vg_lab
[sudo] password for centos:
  Logical volume "lv_data" created.
[centos@localhost ~]$ sudo lvcreate -L 5G -n lv_logs vg_lab
  Logical volume "lv_logs" created.
[centos@localhost ~]$
[centos@localhost ~]$ sudo lvs
  LV      VG     Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  home    cs     -wi-ao---- 18.07g
  root    cs     -wi-ao---- 37.01g
  swap    cs     -wi-ao----  3.91g
  lv_data vg_lab -wi-a-----  5.00g
  lv_logs vg_lab -wi-a-----  5.00g
[centos@localhost ~]$
```

In this case we used a command **sudo lvs** we can confirm that we created a Logical Volume and we verified that the Logical Size is 5G where we allocated it from the Volume Group.

On the **sudo vgs** we can see that the Free space of the Volume Group used 10 GB of its Volume size

```
[centos@localhost ~]$ sudo vgs
  VG     #PV #LV #SN Attr   VSize   VFree
  cs       1   3   0 wz--n- <59.00g      0
  vg_lab   1   2   0 wz--n- <30.00g <20.00g
[centos@localhost ~]$
```

Now that we have completed the components. This is what we will see on the **lsblk** command.

```
[centos@localhost ~]$ lsblk
NAME             MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                8:0    0   30G  0 disk
├─vg_lab-lv_data 253:3    0    5G  0 lvm
└─vg_lab-lv_logs 253:4    0    5G  0 lvm
```

Noticed that sda has two Logical Volumes that are dependent on the vg_lab Volume Group. The vg_lab-lv_data and vg_lab-lv_logs are not partitions, but a Logical block devices created by LVM and they are listed on **/dev/mapper.**

Phase 5. Creating an XFS Filesystem

A Logical Volume only provides storage space. Before Linux can store files on it, a filesystem must be created.

To create a filesystem, we have to decide first what type of filesystem to use. For this phase, we can choose whether to use:

- **XFS**
- **ext4**

Use ext4 if you want the default filesystem for most Linux distribution, supports volumes with sizes up to 1 Exabyte and file sizes up to 16 Terabytes. It is also Reliable, Simple and Easy to repair.

Use xfs if it is built for large systems, very fast and scales extremely well. Very much supported by RHEL distributions including CentOS, AlmaLinux, RockyLinux. It is very much a use case for Servers that provides, Database, Hypervisor, Docker or Fileserver and even Enterprise Application servers, because it can use from 4 - 10TB of storage. For this scenario we are using CentOS which is a RHEL based distribution OS for Linux. So for this case, we will use XFS as a filesystem type.

To create a XFS Filesystem, the command is:

sudo mkfs.xfs /dev/vg_lab/lv_data
sudo mkfs.xfs /dev/vg_lab/lv_logs

```
[centos@localhost ~]$ sudo mkfs.xfs /dev/vg_lab/lv_data
[sudo] password for centos:
meta-data=/dev/vg_lab/lv_data    isize=512    agcount=4, agsize=327680 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=1310720, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
[centos@localhost ~]$ sudo mkfs.xfs /dev/vg_lab/lv_logs
meta-data=/dev/vg_lab/lv_logs    isize=512    agcount=4, agsize=327680 blks
         =                       sectsz=512   attr=2, projid32bit=1
         =                       crc=1        finobt=1, sparse=1, rmapbt=0
         =                       reflink=1    bigtime=1 inobtcount=1 nrext64=0
data     =                       bsize=4096   blocks=1310720, imaxpct=25
         =                       sunit=0      swidth=0 blks
naming   =version 2              bsize=4096   ascii-ci=0, ftype=1
log      =internal log           bsize=4096   blocks=16384, version=2
         =                       sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                   extsz=4096   blocks=0, rtextents=0
[centos@localhost ~]$
```

To verify this, we can use the command **lsblk -f**

```
[centos@localhost ~]$ lsblk -f
NAME             FSTYPE      FSVER    LABEL UUID                                   FSAVAIL FSUSE% MOUNTPOINTS
sda              LVM2_member LVM2 001       Y8UwJ0-1VEE-NmLN-GTCB-qBZh-ZJBZ-tGqiCs
├─vg_lab-lv_data xfs                        7769af24-cf71-4c39-9ed6-aef1ac6664d0
└─vg_lab-lv_logs xfs                        be166e93-95c0-4cbd-904b-f761103babee
```

We can see the FSTYPE on the vg_lab-lv_data and vg_lab-lv_logs now uses xfs. From here, we can now create a file storage and mount the Logical Volumes.

Phase 6. Mounting the Logical Volume

Before the filesystem can be accessed, it must be mounted to a directory. We can create a mount point under the root (**/**) directory. First let's create **/data** and **/logs.**

sudo mkdir /data
sudo mkdir /logs

Mount the Logical Volume.

sudo mount /dev/vg_lab/lv_data /data
sudo mount /dev/vg_lab/lv_logs /logs

Verify that the filesystem has been mounted using

df -h

```
[centos@localhost ~]$ sudo mkdir /data
[sudo] password for centos:
[centos@localhost ~]$ sudo mkdir /logs
[centos@localhost ~]$ sudo mount /dev/vg_lab/lv_data /data
[centos@localhost ~]$ sudo mount /dev/vg_lab/lv_logs /logs
[centos@localhost ~]$
[centos@localhost ~]$ df -h
Filesystem                 Size  Used Avail Use% Mounted on
devtmpfs                   1.8G     0  1.8G   0% /dev
tmpfs                      1.8G     0  1.8G   0% /dev/shm
tmpfs                      725M  9.9M  715M   2% /run
/dev/mapper/cs-root         37G  5.2G   32G  14% /
/dev/nvme0n1p1             960M  441M  520M  46% /boot
/dev/mapper/cs-home         19G  253M   18G   2% /home
tmpfs                      363M   52K  363M   1% /run/user/42
tmpfs                      363M  100K  363M   1% /run/user/1000
/dev/mapper/vg_lab-lv_data  5.0G   68M  4.9G   2% /data
/dev/mapper/vg_lab-lv_logs  5.0G   68M  4.9G   2% /logs
[centos@localhost ~]$
```

As we look on the filesystem, we can now see the Logical Volumes are mounted to /data and /logs directories.

So in this case the Logical Volumes are mounted temporarily. After a reboot, it will no longer be mounted automatically so we need to permanently assign this to **/etc/fstab.** The goal here is that when Linux reads the **/etc/fstab** it will also include the /dev/vg_lab/lv_data and lv_logs to mount on the directories.

When creating a persistent mounting, we can identify the UUID of the filesystem by using **lsblk -f** or **blkid.** Although it is much easier when we use the filepath, as long as the files aren't renamed then using the filepath is fine.

```
[centos@localhost ~]$ sudo cat /etc/fstab

#
# /etc/fstab
# Created by anaconda on Mon Jul  6 19:42:45 2026
#
# Accessible filesystems, by reference, are maintained under '/dev/disk/'.
# See man pages fstab(5), findfs(8), mount(8) and/or blkid(8) for more info.
#
# After editing this file, run 'systemctl daemon-reload' to update systemd
# units generated from this file.
#
/dev/mapper/cs-root     /                       xfs     defaults        0 0
UUID=c92d909a-0b9f-40b1-9180-3b5e41f3b0cf /boot                   xfs     defaults        0 0
/dev/mapper/cs-home     /home                   xfs     defaults        0 0
/dev/mapper/cs-swap     none                    swap    defaults        0 0

# LVM Data and Logs
/dev/vg_lab/lv_data     /data                   xfs     defaults        0 0
/dev/vg_lab/lv_logs     /logs                   xfs     defaults        0 0
[centos@localhost ~]$
```

After adding the Logical Volumes to fstab save and exit the editor and use the command sudo mount -a as if to refresh the mounting of the Logical Volumes.

Phase 7. Expanding the Logical Volume

One of the LVM's greatest advantages is the ability to increase storage capacity without recreating partitions or moving data.

Suppose the **/data** begins running out of available spaces. Since free space still exists inside the Volume Group, we can simply extend the Logical Volume.

We can increase the Logical Volume by 10 GB, and to do that we will use the command:

sudo lvextend -L +5G /dev/vg_lab/lv_data

Noticed that we used 5G to expand the size of the Logical Volume? Currently the size of the lv_data is 5G and adding a +5G will expand to a 10GB size on the lv_data.

```
[centos@localhost ~]$ sudo lvextend -L +5G /dev/vg_lab/lv_data
[sudo] password for centos:
  Size of logical volume vg_lab/lv_data changed from 5.00 GiB (1280 extents) to 10.00 GiB (2560 extents).
  Logical volume vg_lab/lv_data successfully resized.
[centos@localhost ~]$
[centos@localhost ~]$ sudo lvs
  LV      VG     Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  home    cs     -wi-ao---- 18.07g
  root    cs     -wi-ao---- 37.01g
  swap    cs     -wi-ao----  3.91g
  lv_data vg_lab -wi-ao---- 10.00g
  lv_logs vg_lab -wi-ao----  5.00g
[centos@localhost ~]$
```

Noticed how the size of the lv_data had increased to 10 GB, the 5GB had merged with another 5GB and became 10GB on its Logical Volume size. However this does not affect on the filesystem. When we check using **df -h,** we still see that the size remains in 5GB on the lv_data.

```
/dev/mapper/vg_lab-lv_data  5.0G   68M  4.9G   2% /data
/dev/mapper/vg_lab-lv_logs  5.0G   68M  4.9G   2% /logs
[centos@localhost ~]$
```

That is because we only expanded the Logical Volume, not the filesystem itself. To do that we will use the command

sudo xfs_growfs /data

This command expands the size of the filesystem to utilize newly added space from the Logical Volume.

```
/dev/mapper/vg_lab-lv_data   10G  104M  9.9G   2% /data
/dev/mapper/vg_lab-lv_logs  5.0G   68M  4.9G   2% /logs
[centos@localhost ~]$
```

Now we see the changes on the size of the filesystem. It went from 5GB to 10GB, this is because it expanded the size of the filesystem based on the size of the Logical Volume, in this case the **lv_data** that is mounted to the **/data.**

But also there is a shortcut method for this and we can also try doing that on the **lv_logs.** Supposed we use the lvextend command, but the extension goes like this:

sudo lvextend -r -L +5G /dev/vg_lab/lv_logs

The -r option automatically resizes the XFS filesystem after extending the Logical Volume, eliminating the need to run a separate filesystem expansion command.

```
[centos@localhost ~]$ sudo lvs && df -h /dev/mapper/vg_lab-lv_logs
  LV      VG     Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  home    cs     -wi-ao---- 18.07g
  root    cs     -wi-ao---- 37.01g
  swap    cs     -wi-ao----  3.91g
  lv_data vg_lab -wi-ao---- 10.00g
  lv_logs vg_lab -wi-ao---- 10.00g
Filesystem                 Size  Used Avail Use% Mounted on
/dev/mapper/vg_lab-lv_logs   10G  104M  9.9G   2% /logs
[centos@localhost ~]$
```

Noticed that the Logical Volume and the filesystem also expanded to 10GB without using the **sudo xfs_growfs /logs.** This is because -r had resized them simultaneously and it is more efficient than having to expand first the Logical Volume before the Filesystem.

Phase 8. Expanding the Volume Group

Eventually, the Volume Group itself may run out of available storage.

Rather than replacing the existing disk, LVM allows additional physical disks to be added to the existing storage pool.

The second 30 GB disk (**/dev/sdb**) will now be used to expand the Volume Group. The procedure is still the same until in the Volume Group.

Once we created and initialized Physical Volume, now we can assign it to an existing Volume Group, and the command is **vgextend.** The purpose of this is to expand the size of the Volume Group, having two Physical Volumes are in the same Volume Group.

sudo vgextend vg_lab /dev/sdb

```
[centos@localhost ~]$ sudo vgs
  VG     #PV #LV #SN Attr   VSize   VFree
  cs       1   3   0 wz--n- <59.00g      0
  vg_lab   1   2   0 wz--n- <30.00g <10.00g
[centos@localhost ~]$
```

Before we execute the command, this is what the size of vg_lab looks like. Expanding the size by assigning /dev/sdb to the vg_lab**.** This is now the size of vg_lab.

```
[centos@localhost ~]$ sudo vgextend vg_lab /dev/sdb
  Volume group "vg_lab" successfully extended
[centos@localhost ~]$ sudo vgs
  VG     #PV #LV #SN Attr   VSize   VFree
  cs       1   3   0 wz--n- <59.00g     0
  vg_lab   2   2   0 wz--n-  59.99g 39.99g
[centos@localhost ~]$
```

Now as we verify the Volume Group, we've already expanded its size capacity and from there we can further expand the size of the existing Logical Volumes.

```
[centos@localhost ~]$ sudo lvs
  LV      VG     Attr       LSize  Pool Origin Data%  Meta%  Move Log Cpy%Sync Convert
  home    cs     -wi-ao---- 18.07g
  root    cs     -wi-ao---- 37.01g
  swap    cs     -wi-ao----  3.91g
  lv_data vg_lab -wi-ao---- 15.00g
  lv_logs vg_lab -wi-ao---- 20.00g
[centos@localhost ~]$ df -h /dev/mapper/vg_lab-*
Filesystem                Size  Used Avail Use% Mounted on
/dev/mapper/vg_lab-lv_data   15G  140M   15G   1% /data
/dev/mapper/vg_lab-lv_logs   20G  176M   20G   1% /logs
[centos@localhost ~]$
```

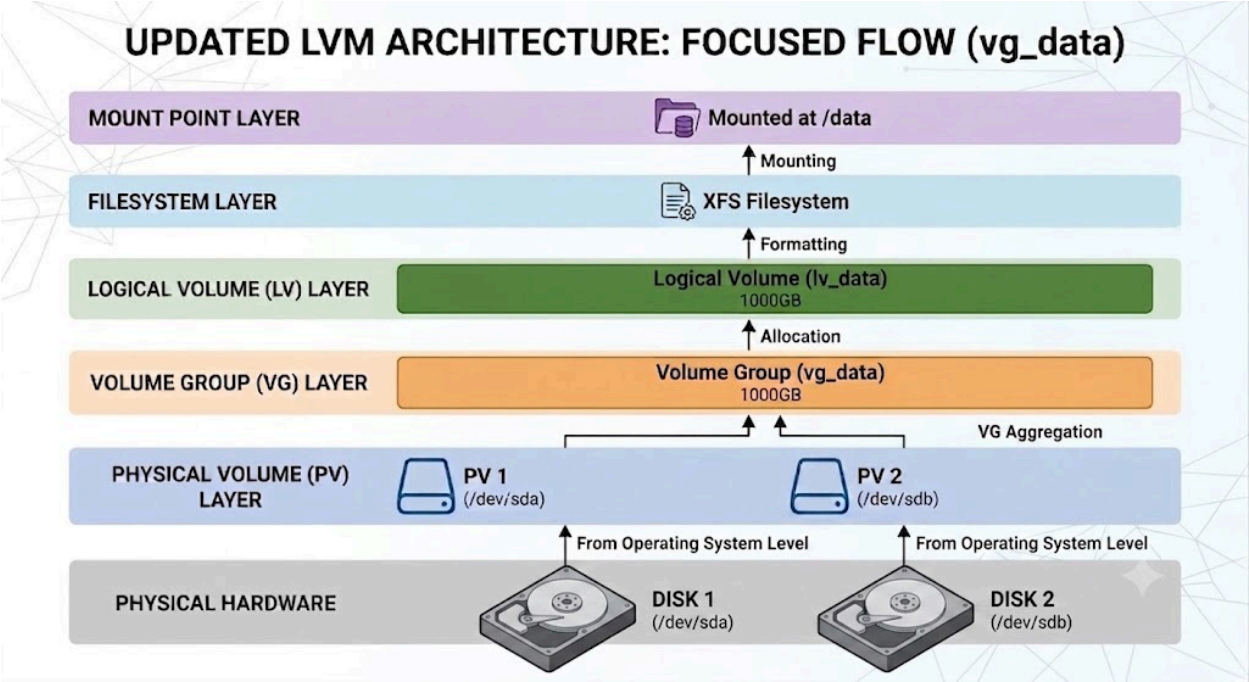
So in this example I was able to expand further the size of the Logical Volumes for lv_data and lv_logs and aswell as their filesystem.

```
[centos@localhost ~]$ sudo vgs
  VG     #PV #LV #SN Attr   VSize   VFree
  cs       1   3   0 wz--n- <59.00g     0
  vg_lab   2   2   0 wz--n-  59.99g 24.99g
[centos@localhost ~]$
```

Now on the Volume Group the size also decreases as the Logical Volumes allocated more capacity from it.

```
[centos@localhost ~]$ sudo pvs
  PV             VG     Fmt  Attr PSize   PFree
  /dev/nvme0n1p2 cs     lvm2 a--  <59.00g     0
  /dev/sda       vg_lab lvm2 a--  <30.00g     0
  /dev/sdb       vg_lab lvm2 a--  <30.00g 24.99g
[centos@localhost ~]$
```

Now the amazing part here is that on the Physical Volume, noticed that the sda already used alot of its size, but looking at the sdb we still have 25GB free size and this already reflected on the Volume Group.



Summary

In this lab, you learned how **Logical Volume Manager (LVM)** abstracts physical storage through multiple layers, making storage management more flexible and scalable than traditional disk partitioning.

The complete workflow consisted of:

1. Adding two additional virtual hard disks to the virtual machine.
2. Creating a Physical Volume (PV) from an empty disk.
3. Adding the Physical Volume to a Volume Group (VG).
4. Allocating storage by creating Logical Volumes (LVs).
5. Creating XFS filesystems on the Logical Volumes.
6. Mounting the Logical Volumes and configuring persistent mounting through `/etc/fstab`.
7. Expanding the Logical Volumes and their XFS filesystems while the system remained online.
8. Expanding the Volume Group by adding a second Physical Volume.

By introducing these abstraction layers, LVM makes storage management significantly more flexible than traditional disk partitioning. As storage requirements grow, administrators can add new physical disks, extend existing Volume Groups, and increase the size of Logical Volumes with minimal downtime and without repartitioning disks or migrating existing data.